Unit I

Introduction to Java

Overview of Object Oriented Programming principles, Importance of Java to the Internet, Byte code, Methods, classes and instances, Data types, arrays, control statements, simple java program, Classes and Objects– constructors, methods, access control, this keyword, overloading methods and constructors, garbage collection, java.lang – String, StringBuffer, StringBuilder, Type wrappers – Integer, Character, Float, Double, Boolean

➢ Overview of C language:

1. C language is known as structure oriented language or procedure oriented language.
2. Employs top-down programming approach where a problem is viewed as a sequence of tasks to be performed.
3. All program code of c can be executed in C++ but converse many not be possible.
4. Function overloading and operator overloading are not possible.
5. Local variables can be declared only at the beginning of the block.
6. Program controls are through jumps and calls to subroutines.
7. Polymorphism, encapsulation and inheritance are not possible. For solving the problems, the problem is divided into a number of modules. Each module is a subprogram.
8. Data abstraction property is not supported by procedure oriented language.
9. Data in procedure oriented language is open and can be accessed by any function.

➢ Difference Between Procedure Oriented Programming (POP) & Object Oriented Programming (OOP):

	Procedure Oriented Programming	Object Oriented Programming
1	program is divided into small partscalled **functions**.	program is divided into parts called **objects**.
2	Importance is not given to **data** but tofunctions as well as **sequence** of actions to be done.	Importance is given to the data rather thanprocedures or functions because it works as a **real world**.
3	follows **Top Down approach**.	OOP follows **Bottom Up approach**.
4	It does not have any access specifier.	OOP has access specifiers named Public, Private, Protected, etc.
5	Data can move freely from function to function in the system.	objects can move and communicate with each other through member functions.
6	To add new data and function in POP is not so easy.	OOP provides an easy way to add new data and function.
7	Most function uses Global data for sharing that can be accessed freely from function to function in the system.	In OOP, data can not move easily from function to function,it can be kept public or private so we can control the access of data.
8	It does not have any proper way for hiding	OOP provides Data Hiding so provides more

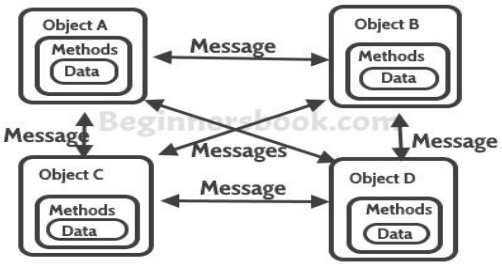
	data so it is less secure.	security.
9	Overloading is not possible.	In OOP, overloading is possible in the form of Function Overloading and Operator Overloading.
10	Example of Procedure Oriented Programming are : C, VB, FORTRAN, Pascal.	Example of Object Oriented Programming are : C++, JAVA, VB.NET, C#.NET.

➢ Object Oriented Programming :

Object Oriented Programming is an approach that provides a way of modularizing programs by creating partitioned memory area for both data and functions that can be used as template for creating copies on demand.

➢ Features of OOPs:

1. **Object:** Any entity that has state and behaviour is known as an object. For example: chair, pen, table, keyboard, bike etc. It can be physical and logical.
   Example: A dog is an object because it has states i.e. color, name, breed etc. as well as behaviours i.e. wagging the tail, barking, eating etc.
2. **Class:** Collection of objects is called class. It is a logical entity. A class can also be defined as a blueprint from which you can create an individual object.
3. **Data Encapsulation:** Wrapping of data and methods into single unit called class is known as Data Encapsulation.
4. **Data Abstraction:** Abstraction refers to the act of representing essential features without including background details or explanations.
5. **Polymorphism:** Ability to take more than one form is known as Polymorphism. ex: area(l,b) area(r)
6. **Inheritance:** Inheritance is the process by which one class derives the properties of another class.
7. **Dynamic binding:** Binding refers to linking of procedure call to the code to be executed in response to the call. Dynamic binding (or late binding) means the code associated with a given procedure call in not knownuntil the time of call at run time.
8. **Message passing:** An object oriented program consists of set of object that communicate with each other. Objects communicates with each other by sending and receiving information. A message for an object is a request for execution of a procedure and therefore invoke the function that is called for an object and generates result.



If any language fallows the OOPS concepts that language we call it as object oriented language

- **Benefits (Objectives) of Object Oriented Programming (OOPs):**
  1. **Reusability:** In OOP‟ s programs functions and modules that are written by a user can be reused byother users without any modification.
  2. **Inheritance**: Through this we can eliminate redundant code and extend the use of existing classes.
  3. **Data Hiding:** The programmer can hide the data and functions in a class from other classes. It helps the programmer to build the secure programs.
  4. **Reduced complexity of a problem:** The given problem can be viewed as a collection of different objects. Each objectis responsible for a specific task. The problem is solved by interfacing the objects. This technique reduces the complexity of the program design.
  5. **Easy to Maintain and Upgrade**: OOP makes it easy to maintain and modify existing code as new objects can be created with small differences to existing ones. Software complexity can be easily managed.
  6. **Message Passing:** The technique of message communication between objects makes the interfacewith external systems easier.
  7. **Modifiability:** it is easy to make minor changes in the data representation or the procedures in anOO program. Changes inside a class do not affect any other part of a program, since the only public interface that the external world has to a class is through the use of methods.

- **History of Java** James Gosling, Patrick Naughton and Mike Sheridan initiated the Java language project in 1991. Team of sun engineers designed for small, embedded systems in electronic appliances like set-top boxes. Initially it was called "Greentalk" later it was called OAK. Java is an open source software produced by Sun micro system under the terms of the GNU General Public License (GPL) .
- **Importance Of Java in Internet:**
  - The Internet helped catapult Java to the forefront of programming, and Java, in turn, has had a profound effect on the Internet. The reason for this is quite simple: Java expands the universe of objects that can move about freely in cyberspace. In a network, two very broad categories of objects are transmitted between the server and your Personal computer: passive information and dynamic, active programs.
  - When you read your e-mail, you are viewing passive data. Even when you download a program, the program's code is still only passive data until you execute it.
  - However, a second type of object can be transmitted to your computer: a dynamic, self-executing program. Such a program is an active agent on the client computer, yet is initiated by the server. For example, a program might be provided by the server to display properly the data that the server is sending.
  - As desirable as dynamic, networked programs are, they also present serious problems in the areas of security and portability. Prior to Java, cyberspace was effectively closed to half the entities that now live there.
  - As you will see, Java addresses those concerns and, by doing so, has opened the door to an exciting new form of program: the applet.

- In a network, two very broad categories of objects are transmitted between the server and your personal computer: passive information and dynamic, active programs. For example, when you read your e-mail, you are viewing passive data. Even when you download a program, the program's code is still only passive data until you execute it. However, a second type of object can be transmitted to your computer: a dynamic, self-executing program
- In a network, two very broad categories of objects are transmitted between the server and your personal computer: passive information and dynamic, active programs. For example, when you read your e-mail, you are viewing passive data. Even when you download a program, the program's code is still only passive data until you execute it. However, a second type of object can be transmitted to your computer: a dynamic, self-executing program
- **Java Applets and Applications:** An application is a program that runs on your computer. It is Java's ability to create applets that makes it important. An applet is an application designed to be transmitted over the Internet and executed by a Java-compatible Web browser.
- Byte Code java's magic for solve above to problems(security and portability): The output of a Java compiler is not executable code. Rather, it is bytecode. Bytecode is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called the Java Virtual Machine (JVM). That is, in its standard form, the JVM is an interpreter for bytecode only the JVM needs to be implemented for each platform. Once the run-time package exists for a given system, any Java program can run on it. Remember, although the details of the JVM will differ from platform to platform, all interpret the same Java bytecode.
- The Java words Although the fundamental forces that necessitated the invention of Java are portability and security, other factors also played an important role
  - **Object Oriented** – Java implements basic concepts of Object oriented programming System (OOPS) ie Object, Class, Inheritance, Polymorphism, Abstraction, Encapsulation. In Java, everything is an Object. Java can be easily extended since it is based on the Object model.
  - **Platform Independent** – Unlike many other programming languages including C and C++, when Java is compiled, it is not compiled into platform specific machine, rather into platform independent byte code. This byte code is distributed over the web and interpreted by the Virtual Machine (JVM) on whichever platform it is being run on.
  - **Simple** – Java fallows the basic Syntax of C,C++. If you understand the basic concept of OOPS then it is easy to master in java.
  - **Secure** – With Java's secure feature it enables to develop virus-free, tamper-free systems. Authentication techniques are based on public key encryption.
  - **Architecture-neutral** – Java compiler generates an architecture-neutral object file format, which makes the compiled code executable on many processors, with the presence of Java runtime system.
  - **Portable** – Being architecture-neutral and having no implementation dependent aspects of the specification makes Java portable. Compiler in Java is written in ANSI C with a clean portability boundary, which is a POSIX subset.

- **Robust** – Java makes an effort to eliminate error prone situations by emphasizing mainly on compile time error checking and runtime checking.
- **Multithreaded** – With Java's multithreaded feature in java we can write programs that can perform many tasks simultaneously. This design feature allows the developers to construct interactive applications that can run smoothly.
- **Interpreted** – Java byte code is translated on the fly to native machine instructions and is not stored anywhere. The development process is more rapid and analytical since the linking is an incremental and light-weight process.
- **High Performance** – With the use of Just-In-Time compilers, Java enables high performance.
- **Distributed** – Java is designed for the distributed environment of the internet.
- **Dynamic** – Java is considered to be more dynamic than C or C++ since it is designed to adapt to an evolving environment. Java programs can carry extensive amount of run-time information that can be used to verify and resolve accesses to objects on run-time.

➢ JVM (Java Virtual Machine):

JVM (Java Virtual Machine) is an abstract machine. It is a specification that provides runtime environment in which java bytecode can be executed. JVMs are available for many hardware and software platforms (i.e. JVM is platform dependent).

What is JVM: It is:

1. A specification where working of Java Virtual Machine is specified. But implementation provider is independent to choose the algorithm. Its implementation has been provided by Sun and other companies.
2. An implementation Its implementation is known as JRE (Java Runtime Environment).
3. Runtime Instance Whenever you write java command on the command prompt to run the java class, an instance of JVM is created.

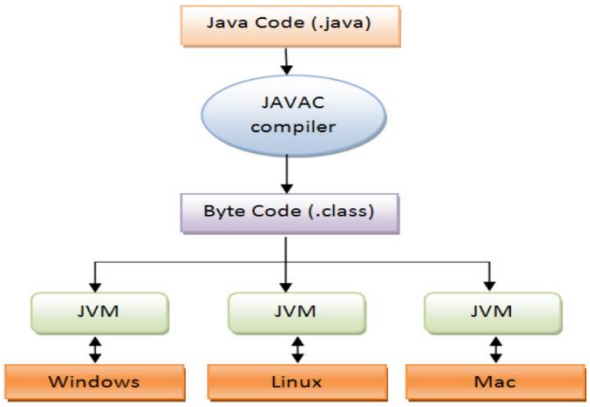
What it does:

The JVM performs following operation:

- Loads code
- Verifies code
- Executes code
- Provides runtime environment

JVM provides definitions for the:

- Memory area
- Class file format
- Register set
- Garbage-collected heap
- Fatal error reporting etc.



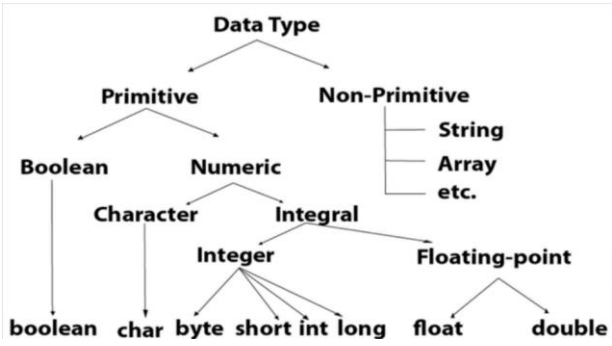
Java's Magic: The Bytecode output of a Java compiler is not executable code. Rather, it is bytecode. Bytecode is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called the Java Virtual Machine (JVM).

Java complier translates the java source code into byte code or intermediate code, not the executable file .JVM take the byte code and convert into executable code corresponding to Operating system Because of the above feature java is portable

➢ Data Types in JAVA:

Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:

1. **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
2. **Non-primitive data types:** The non-primitive data types include Classes, Interfaces, and Arrays.



Data Type	Default Value	Default size
boolean	false	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
float	0.0f	4 byte
double	0.0d	8 byte

Primitive Data Types:
There are eight primitive data types supported by Java. Primitive data types are predefined by the language and named by a keyword. Let us now look into the eight primitive data types in detail.

byte

- Byte data type is an 8-bit signed two's complement integer
- Minimum value is -128 (-2^7)
- Maximum value is 127 (inclusive)(2^7 -1)
- Default value is 0
- Byte data type is used to save space in large arrays, mainly in place of integers, since a byte is four times smaller than an integer.
- Example: byte a = 100, byte b = -50

short

- Short data type is a 16-bit signed two's complement integer
- Minimum value is -32,768 (-2^15)
- Maximum value is 32,767 (inclusive) (2^15 -1)
- Short data type can also be used to save memory as byte data type. A short is 2 times smaller than an integer
- Default value is 0.
- Example: short s = 10000, short r = -20000

int

- Int data type is a 32-bit signed two's complement integer.
- Minimum value is - 2,147,483,648 (-2^31)
- Maximum value is 2,147,483,647(inclusive) (2^31 -1)
- Integer is generally used as the default data type for integral values unless there is a concern about memory. • The default value is 0
- Example: int a = 100000, int b = -200000

long

- Long data type is a 64-bit signed two's complement integer
- Minimum value is -9,223,372,036,854,775,808(-2^63)
- Maximum value is 9,223,372,036,854,775,807 (inclusive)(2^63 -1)
- This type is used when a wider range than int is needed
- Default value is 0L
- Example: long a = 100000L, long b = -200000L

float

- Float data type is a single-precision 32-bit IEEE 754 floating point
- Float is mainly used to save memory in large arrays of floating point numbers
- Default value is 0.0f
- Float data type is never used for precise values such as currency
- Example: float f1 = 234.5f

double

- double data type is a double-precision 64-bit IEEE 754 floating point
- This data type is generally used as the default data type for decimal values, generally the default choice
- Double data type should never be used for precise values such as currency
- Default value is 0.0d
- Example: double d1 = 123.4

boolean

- boolean data type represents one bit of information
- There are only two possible values: true and false
- This data type is used for simple flags that track true/false conditions
- Default value is false
- Example: boolean one = true

char

- char data type is a single 16-bit Unicode character
- Minimum value is '\u0000' (or 0)
- Maximum value is '\uffff' (or 65,535 inclusive)
- Char data type is used to store any character
- Example: char letterA = 'A'

➢ Byte code:

Java bytecode is the instruction set for the Java Virtual Machine. Byte code is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called the Java Virtual Machine (JVM). That is, in its standard form, the JVM is an interpreter for byte code. Java bytecode is the result of the compilation of a Java program, an intermediate representation of that program which is machine independent.

How does it works: When we write a program in Java, firstly, the compiler compiles that program and a bytecode is generated for that piece of code. When we wish to run this .class file on any other platform, we can do so. After the first compilation, the bytecode generated is now run by the Java Virtual Machine and not the processor in consideration. This essentially means that we only need to have basic java installation on any platforms that we want to run our code on. Resources required to run the bytecode are made available by theJava Virtual Machine, which calls the processor to allocate the required resources. JVM's are stack-based so they stack implementation to read the codes.

➢ JAVA Program Structure:

1. **Documentation Section:** It includes the comments to tell the program's purpose. It improves the readability of the program.
   Explination:It includes the comments that improve the readability of the program. A comment is a non-executable statement that helps to read and understand a program especially when your programs get more complex. It is simply a message that exists only for the programmer and is ignored by the compiler. A good program should include comments that describe the purpose of

the program, author name, date and time of program creation. This section is optional and comments may appear anywhere in the program.

Java programming language supports three types of comments.

- **Single line (or end-of line) comment:** It starts with a double slash symbol (//) and terminates at the end of the current line. The compiler ignores everything from // to the end of the line. For example:

  // Calculate sum of two numbers
- **Multiline Comment:** Java programmer can use C/C++ comment style that begins with delimiter /* and ends with */. All the text written between the delimiter is ignored by the compiler. This style of comments can be used on part of a line, a whole line or more commonly to define multi-line comment. For example.

  /*calculate sum of two numbers */

  Comments cannot be nested. In other words, you cannot comment a line that already includes traditional comment. For example,

  /* x = y /* initial value */ + z; */ is wrong.
- **Documentation comments:** This comment style is new in Java. Such comments begin with delimiter /** and end with */. The compiler also ignores this type of comments just like it ignores comments that use / * and */. The main purpose of this type of comment is to automatically generate program documentation. The java doc tool reads these comments and uses them to prepare your program's documentation in HTML format. For example.

/**The text enclosed here will be part of program documentation */

2. **Package Statement:** It includes statement that provides a package declaration.

   Explination:Java allows you to group classes in a collection known as package. A package statement includes a statement that provides a package declaration. It must appear as the first statement in the source code file before any class or interface declaration. This statement is optional. For example: Suppose you write the following package declaration as the first statement in the source code file.

   package employee;

   This statement declares that all classes and interfaces defined in this source file are part of the employee package. Only one package declaration can appear in the source file.
3. **Import statements:** It includes statements used for referring classes and interfaces that are declared in other packages.

   Explination:Java contains many predefined classes that are stored into packages. In order to refer these standard predefined classes in your program, you need to use fully qualified name (i.e. Packagename.className). But this is a very tedious task as one need to retype the package path name along with the classname. So a better alternative is to use an import statement.

   An import statement is used for referring classes that are declared in other packages. The import statement is written after a package statement but before any class definition. You can import a specific class or all the classes of the package. For example : If you want to import Date class of java.util package using import statement then write

```
import java.util.Date;
```

This statement allows the programmer to use the simple classname Date rather than fully qualified classname java.util.Date in the code.

Unlike package statement, you can specify more than one import statement in your program.

For example:

```
Import java.util.Date; /* imports only the Date class in java.util package */
import java.applet.*; // imports all the classes in java applet
                      // package
```

4. **Interface Section:** It is similar to a class but only includes constants, method declaration.

   Explination: In the interface section, we specify the interfaces. An interface is similar to a class but contains only constants and method declarations. Interfaces cannot be instantiated. They can only be implemented by classes or extended by other interfaces. It is an optional section and is used when we wish to implement multiple inheritance feature in the program.

```
interface stack
{
void push(int item); // Insert item into stack
int pop(); // Delete an item from stack
}
```

5. **Class Section:** It describes information about user defines classes present in the program. Every Java program consists of at least one class definition. This class definition declares the main method. It is from where the execution of program actually starts.

   Explination:The Class section describes the information about user-defined classes present in the program. A class is a collection of fields (data variables) and methods that operate on the fields. Every program in Java consists of at least one class, the one that contains the main method. The main () method which is from where the execution of program actually starts and follow the statements in the order specified.

   The main method can create objects, evaluate expressions, and invoke other methods and much more. On reaching the end of main, the program terminates and control passes back to the operating system.

   The class section is mandatory.

   After discussing the structure of programs in Java, we shall now discuss a program that displays a string Hello Java on the screen.

- **Simple JAVA Program:**

```
class Example
{
public static void main(String args[])
{
System.out.println("Hello Word…!");
// println() is the console output statement
}
```

}

--- Name of the source file is the name of the class having main() method by convention

--- Class names start with capital letter for each word

--- Method names start with small case letter for first word and capital letter for each subsequent word

--- File extension for source file is .java

--- JDK(Java Development Kit) is the software toolkit containing Java Compiler(javac) and Java Interpreter(JVM-java)

--- Current JDK version is jdk20.0

--- System.out.println() prints a message onto the console output screen

--- System.out.print() doesn't add a newline, while println() adds a newline

--- Comment style is /*…….*/ for multiline comment and

//……. for single line comment

--- Command for compiling source file is

javac filename

--- Compile time errors are reported, if any

--- After compiling a source file, class files containing bytecode is created for each class

--- The class file for the class having main() method is then sent for interpreting and execution.

--- Command for interpreting and executing a class file is

java classname

Compiling:

javac Example.java

Interpreting and Execution:

java Example

Output:

Hello Word…!

- **Array:**

Array is group of elements of similar types occupying contiguous memory locations

Advantage of Java Array :

- Code Optimization
- Random access

Disadvantage of Java Array is it has fixed size it cannot grow

There are two types of array in java .

- Single Dimensional Array
- Multidimensional Array

Single Dimensional Array Syntax to Declare an Array in java

1. datatype[] arrayname (or)
2. datatype []arrayname; (or)
3. datatype arrayname[];

Initialization and instantiate of an Array in java

1. int a[]={33,3,4,5}; // static initilaziation
2. arrayname=new datatype[size]; // instantiate

Example Let's see the simple example of java array, where we are going to declare, instantiate, initialize and traverse an array.

```
class Testarray {
  public static void main(String args[]) {
    int a[] = new int[5]; //declaration and instantiation
    a[0] = 10;
    a[1] = 20;
    a[2] = 70;
    a[3] = 40;
    a[4] = 50;
    for (int i = 0; i < a.length; i++) //length is the property of array
      System.out.println(a[i]);
  }
}
```

This would produce the following result:

```
10
20
70
40
50
```

Multidimensional array data is stored in row and column based index (also known as matrix form).

Syntax to Declare Multidimensional Array in java

1. dataType[][] arrayRefVar; (or)
2. dataType [][]arrayRefVar; (or)
3. dataType arrayRefVar[][]; (or)
4. dataType []arrayRefVar[];

instantiate Multidimensional Array in java

```
int[][] arr=new int[row][column]; // declaration and instantiation
```

initialize Multidimensional Array in java

```
int arr[][]={{1,2,3},{2,4,5},{4,4,5}}; // declaration and initialization
```

Example:

```
class TestArray {
  public static void main(String args[]) {
    //declaring and initializing 2D array
    int arr[][] = {{1,2,3},{2,4,5},{4,4,5}};
    //printing 2D array
    for (int i = 0; i < 3; i++) {
      for (int j = 0; j < 3; j++) {
        System.out.print(arr[i][j] + " ");
      }
      System.out.println();
    }
  }
}
```

This would produce the following result:

1 2 3
2 4 5
4 4 5

➢ **Java Literals**:

A literal is a source code representation of a fixed value. Literals can be assigned to any primitive type variable.
byte a = 68;
char a = 'A'
byte, int, long, and short can be expressed in decimal(base 10), hexadecimal(base 16) or octal(base 8) number systems as well.

Prefix 0 is used to indicate octal, and prefix 0x indicates hexadecimal when using these number systems for literals. For example –
int decimal = 100; int octal = 0144; int hexa = 0x64;

String literals in Java are specified like they are in most other languages by enclosing a sequence of characters between a pair of double quotes.
Examples of string literals are –
Example
"Hello World" "two\nlines" "\"This is in quotes\""

String and char types of literals can contain any Unicode characters. For example – char a = '\u0001'; String a = "\u0001";

➢ **Operators in java**:

Operator in java is a symbol that is used to perform operations. For example: +, -, *, / etc.
There are many types of operators in java which are given below:

- Unary Operator,
- Arithmetic Operator,
- shift Operator,
- Relational Operator,
- Bitwise Operator,
- Logical Operator,
- Ternary Operator and
- Assignment Operator.

➢ **Control Statements in JAVA:**

Java compiler executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, Java provides statements that can be used to control the flow of Java code. Such statements are called control flow statements. It is one of the fundamental features of Java, which provides a smooth flow of program.
Java provides three types of control flow statements.

Decision Making Statements:

There are two types of decision making statements in Java. They are:

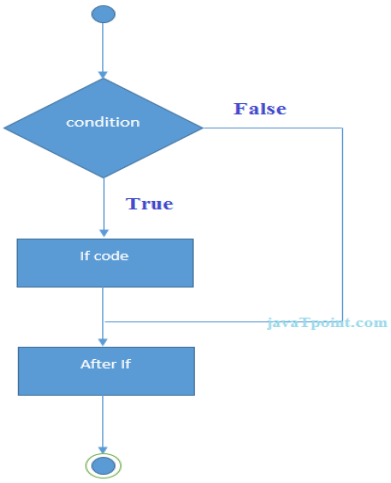
- if statements
- switch statements

The if Statement:

An if statement consists of a Boolean expression followed by one or more statements. **Syntax**: The syntax of an if statement is:

```
if(Boolean_expression) {
//Statements will execute if the Boolean expression is true
}
```

If the Boolean expression evaluates to true, then the block of code inside the if statement will be executed. If not, the first set of code after the end of the if statement(after the closing curly brace) will be executed.



Example:

```
public class Student {
   public static void main(String[] args) {
      int x = 10;
      int y = 12;
      if (x + y > 20) {
         System.out.println("x + y is greater than 20");
      }
   }
}
```

This would produce the following result:

```
x + y is greater than 20
```

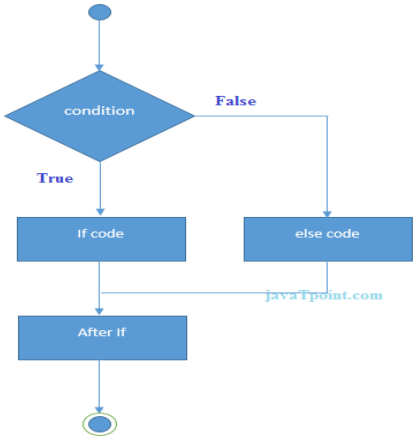
The if...else Statement:

An if statement can be followed by an optional else statement, which executes when the Boolean expression is false.

Syntax: The syntax of an if...else is:

```
if(Boolean_expression){
//Executes when the Boolean expression is true
}
```

```
else {
//Executes when the Boolean expression is false
}
```



Example:

```
public class Student {
   public static void main(String[] args) {
      int x = 10;
      int y = 12;
      if (x + y < 10) {
         System.out.println("x + y is less than     10");
      } else {
         System.out.println("x + y is greater than 20");
      }
   }
}
```

This would produce the following result:

```
x + y is greater than 20
```

- **The if...else if...else Statement:**

An if statement can be followed by an optional else if...else statement, which is very useful to test various conditions using single if...else if statement. When using if, else if , else statements there are few points to keep in mind.

- An if can have zero or one else's and it must come after any else if's.
- An if can have zero to many else if's and they must come before the else.
- Once an else if succeeds, none of the remaining else if's or else's will be tested.

Syntax: The syntax of an if...else is:

```
if(Boolean_expression1){
//Executes when the Boolean expression 1 is true
}
elseif(Boolean_expression2){
//Executes when the Boolean expression 2 is true
}
```

```
elseif(Boolean_expression3){
//Executes when the Boolean expression 3 is true
}else {
//Executes when the none of the above condition is true.
}
```

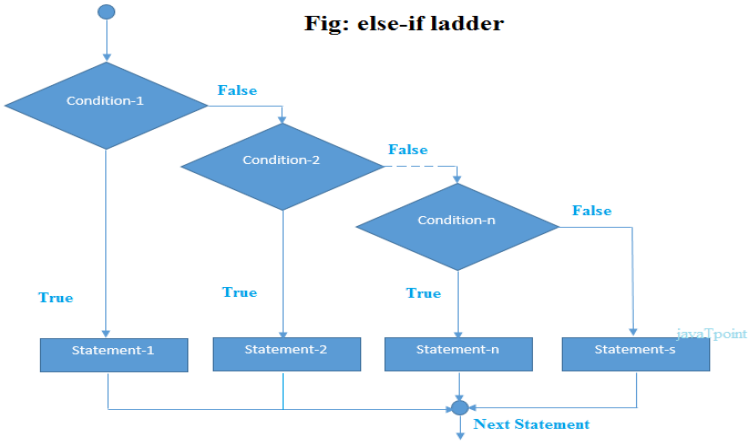


Fig: else-if ladder

Example:

```
public class Student {
  public static void main(String[] args) {
    String city = "Delhi";
    if (city == "Meerut") {
      System.out.println("city is meerut");
    } else if (city == "Noida") {
      System.out.println("city is noida");
    } else if (city == "Agra") {
      System.out.println("city is agra");
    } else {
      System.out.println(city);
    }
  }
}
```

This would produce the following result:

```
Delhi
```

- **Nested if...else Statement**:

It is always legal to nest if-else statements which means you can use one if or else if statement inside another if or else if statement.

Syntax: The syntax for a nested if...else is as follows:

```
if(Boolean_expression1){
//Executes when the Boolean expression 1 is true
if(Boolean_expression2){
//Executes when the Boolean expression 2 is true
}
}
```

You can nest else if...else in the similar way as we have nested if statement.

Example:

```
public class Test {
   public static void main(String args[]) {
      int x = 30;
      int y = 10;
      if (x == 30) {
         if (y == 10) {
            System.out.print("X = 30 and Y = 10");
         }
      }
   }
```

This would produce the following result:

X = 30 and Y = 10

- **The switch Statement:** A switch statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each case.

Syntax: The syntax of enhanced for loop is:

```
switch(expression){
case value : //Statements break;
//optional case value :
//Statements
break;
//optional
//You can have any number of case statements.
default://Optional
//Statements
}
```

The following rules apply to a switch statement:

- The variable used in a switch statement can only be a byte, short, int, or char.
- You can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.
- The value for a case must be the same data type as the variable in the switch and it must be a constant or a literal.
- When the variable being switched on is equal to a case, the statements following that case will execute until a break statement is reached.
- When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
- Not every case needs to contain a break. If no break appears, the flow of control will fall through to subsequent cases until a break is reached.
- A switch statement can have an optional default case, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true. No break is needed in the default case.

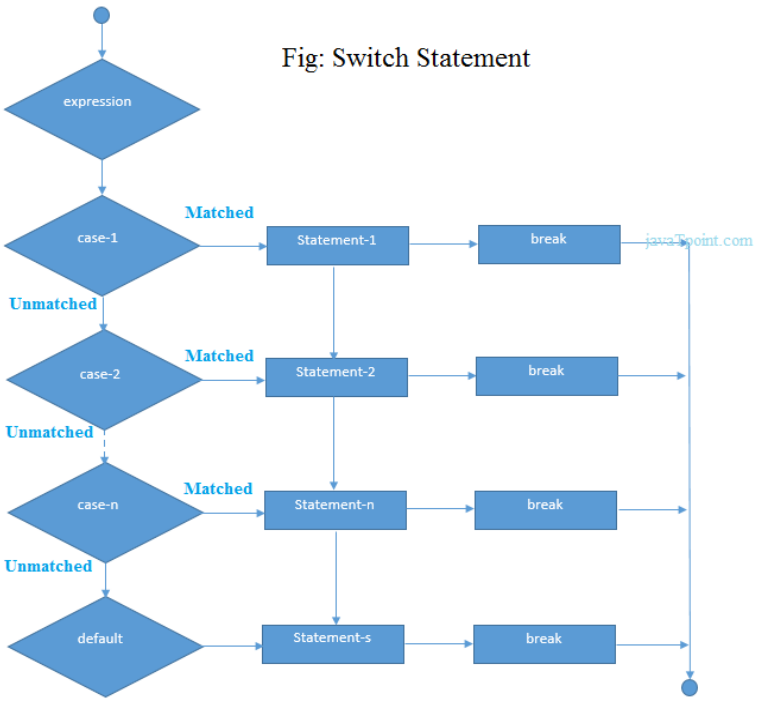


Fig: Switch Statement

Example:

```
public class Student implements Cloneable {
   public static void main(String[] args) {
      int num = 2;
      switch (num) {
         case 0:
            System.out.println("number is 0");
            break;
         case 1:
            System.out.println("number is 1");
            break;
         default:
            System.out.println(num);
      }
   }
}
```

This would produce the following result:

2

- **Loop Control**:

There may be a situation when we need to execute a block of code several number of times and is often referred to as a loop. Java has very flexible three looping mechanisms. You can use one of the following three loops:

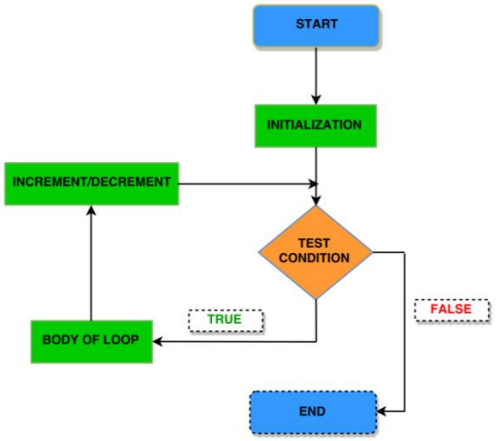
- while Loop
- do...while Loop
- for Loop

  - **The while Loop:** A while loop is a control structure that allows you to repeat a task a certain number of times.

    Syntax: The syntax of a while loop is:

```
while(Boolean_expression) {
//Statements }
```

When executing, if the boolean_expression result is true, then the actions inside the loop will be executed. This will continue as long as the expression result is true. Here, key point of the while loop is that the loop might not ever run. When the expression is tested and the result is false, the loop body will be skipped and the first statement after the while loop will be executed.



Example:

```
public class Calculation {
   public static void main(String[] args) {
      int i = 0;
      System.out.println("Printing the list of first 10 even numbers \n");
      while (i <= 10) {
         System.out.println(i);
         i = i + 2;
      }
   }
}
```

This would produce the following result:

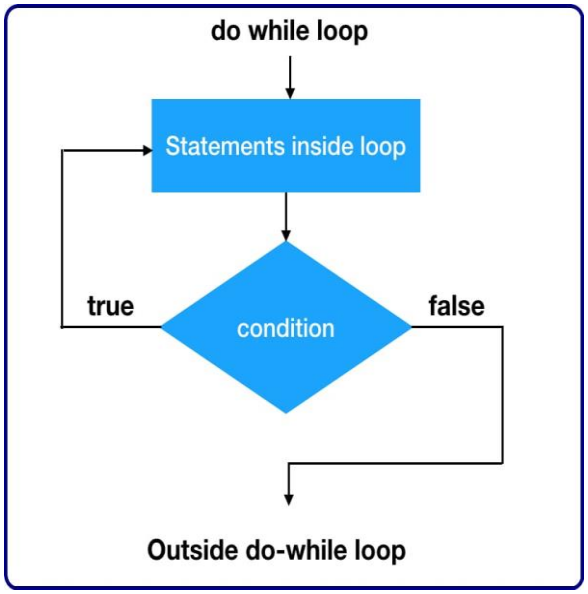
```
Printing the list of first 10 even numbers
0
2
4
6
8
10
```

- **The do...while Loop:** A do...while loop is similar to a while loop, except that a do...while loop is guaranteed to execute at least one time.

Syntax: The syntax of a do...while loop is:

```
do {
//Statements
```

} while(Boolean_expression);



Example:

```
public class Calculation {
   public static void main(String[] args) {
      int i = 0;
      System.out.println("Printing the list of first 10 even numbers \n");
      do {
         System.out.println(i);
         i = i + 2;
      } while (i <= 10);
   }
}
```

This would produce the following result:

```
Printing the list of first 10 even numbers
0
2
4
6
8
10
```

- **The for Loop:** A for loop is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times. A for loop is useful when you know how many times a task is to be repeated.

Syntax: The syntax of a for loop is:

```
for(initialization;Boolean_expression; update) {
 //Statements
}
```

Here is the flow of control in a for loop:

- The initialization step is executed first, and only once. This step allows you to declare and initialize any loop control variables. You are not required to put a statement here, as long as a semicolon appears.
- Next, the Boolean expression is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and flow of control jumps to the next statement past the for loop.
- After the body of the for loop executes, the flow of control jumps back up to the update statement. This statement allows you to update any loop control variables. This statement can be left blank, as long as a semicolon appears after the Boolean expression.
- The Boolean expression is now evaluated again. If it is true, the loop executes and the process repeats itself (body of loop, then update step, then Boolean expression). After the Boolean expression is false, the for loop terminates.

Example:

```
public class Calculattion {
   public static void main(String[] args) {
      int sum = 0;
      for (int j = 1; j <= 10; j++) {
         sum = sum + j;
      }
      System.out.println("The sum of first 10 natural numbers is " +
sum);
   }
}
```

This would produce the following result:

```
The sum of first 10 natural numbers is 55
```

Jump Statements:

Jump statements are used to transfer the control of the program to the specific statements. In other words, jump statements transfer the execution control to the other part of the program. There are two types of jump statements in Java, i.e., break and continue.

- **The break Keyword**: The break keyword is used to stop the entire loop. The break keyword must be used inside any loop or a switch statement. The break keyword will stop the execution of the innermost loop and start executing the next line of code after the block.

Syntax: The syntax of a break is a single statement inside any loop:

```
break;
```

Example:

```
public class BreakExample {
   public static void main(String[] args) {
      for (int i = 0; i <= 10; i++) {
         System.out.println(i);
         if (i == 6) {
            break;
```

```
            }
          }
        }
      }
```

This would produce the following result:

```
0
1
2
3
4
5
6
```

- **The continue Keyword**: The continue keyword can be used in any of the loop control structures. It causes the loop to immediately jump to the next iteration of the loop.
  - In a for loop, the continue keyword causes flow of control to immediately jump to the update statement.
  - In a while loop or do/while loop, flow of control immediately jumps to the Boolean expression.

Syntax: The syntax of a continue is a single statement inside any loop:

```
continue;
```

Example:

```
public class Test {
   public static void main(String args[]) {
      int[] numbers = {10, 20, 30,40, 50};
      for (int x: numbers) {
         if (x == 30) {
            continue;
         }
         System.out.print(x);
         System.out.print("\n");
      }
   }
}
```

This would produce the following result:

```
10
20
40
50
```

- **Methods**:

A Java method is a collection of statements that are grouped together to perform an operation. When you call the System.out.println method, for example, the system actually executes several statements in order to display a message on the console. Now you will learn how to create your own methods with or without return values, invoke a

method with or without parameters, overload methods using the same names, and apply method abstraction in the program design.

Method definition consists of a method header and a method body. The same is shown below:

```
modifier returnType nameOfMethod (Parameter List) {
// method body
}
```

The syntax shown above includes:

- modifier: It defines the access type of the method and it is optional to use.
- returnType: Method may return a value.
- nameOfMethod: This is the method name. The method signature consists of the method name and the parameter list.
- Parameter List: The list of parameters, it is the type, order, and number of parameters of a method. These are optional, method may contain zero parameters.
- method body: The method body defines what the method does with statements.

Creating Method:

Considering the following example to explain the syntax of a method:

```
public static int funcName(int a, int b) {
 // body
}
```

Here,

- public static : modifier.
- int: return type
- funcName: function name
- a, b: formal parameters
- int a, int b: list of parameters

Methods are also known as Procedures or Functions:

- Procedures: They don't return any value.
- Functions: They return value.

Example: Here is the source code of the above defined method called max(). This method takes two parameters num1 and num2 and returns the maximum between the two:

```
/* the snippet returns the minimum between two numbers */
public static int minFunction(int n1, int n2) {
        int min;
        if (n1 > n2)
        min = n2;
        else
        min = n1;
        return min;
}
```

Method Calling:

For using a method, it should be called. There are two ways in which a method is called i.e. method returns a value or returning nothing (no return value). The process of method calling is simple. When a program invokes a method, the program control gets transferred to the called method. This called method then returns control to the caller in two conditions, when:

- return statement is executed.
- reaches the method ending closing brace.

Example: Following is the example to demonstrate how to define a method and how to call it:

```
public class ExampleMinNumber {
  /** returns the minimum of two numbers */
  public static int minFunction(int n1, int n2) {
    int min;
    if (n1 > n2)
      min = n2;
    else
      min = n1;
    return min;
  }
  public static void main(String[] args) {
    int a = 11;
    int b = 6;
    int c = minFunction(a, b);
    System.out.println("Minimum Value = " + c);
  }
}
```

This would produce the following result:

```
Minimum value = 6
```

➢ Object & Classes:

- **Object** - Objects have states and behaviors. Example: A dog has statescolor, name, breed as well as behaviors -wagging, barking, eating. An object is an instance of a class.
- **Class** - A class can be defined as a template/blue print that describes the behaviors/states that object of its type support.

Objects in Java:

Let us now look deep into what are objects. If we consider the real-world we can find many objects around us, Cars, Dogs, Humans, etc. All these objects have a state and behavior. If we consider a dog, then its state is - name, breed, color, and the behavior is - barking, wagging, running If you compare the software object with a real world object, they have very similar characteristics. Software objects also have a state and behavior. A software object's state is stored in fields and behavior is shown via methods. So in software development, methods operate on the internal state of an object and the object to-object communication is done via methods. **Classes in Java:**

A class is a blue print from which individual objects are created. A sample of a class is given below:

```
public class Dog {
   String breed;
   int age;
   String color;
   void barking() {}
   void hungry() {}
   void sleeping() {}
}
```

A class can contain any of the following variable types.

- **Local variables:** Variables defined inside methods, constructors or blocks are called local variables. The variable will be declared and initialized within the method and the variable will be destroyed when the method has completed.
- **Instance variables:** Instance variables are variables within a class but outside any method. These variables are instantiated when the class is loaded. Instance variables can be accessed from inside any method, constructor or blocks of that particular class.
- **Class variables:** Class variables are variables declared within a class, outside any method, with the static keyword.

A class can have any number of methods to access the value of various kinds of methods. In the above example, barking(), hungry() and sleeping() are methods. Below mentioned are some of the important topics that need to be discussed when looking into classes of the Java Language.

Constructors:

- Constructor is a special type of method that initializes instance variables of a class, when an object is created.
- It has same name as that of the class itself.
- It has no return type, even void. Because it returns the object itself.

Example:

```
class Sample {
   int x;
   Sample(int a) // constructor
   {
      x = a;
   }
   void show() {
      System.out.println("x=" + x);
   }
}
class Demo {
   public static void main(String args[]) {
        Sample s = new Sample(5); // calling constructor
        s.show();
        }
     }
```

This would produce the following result:

```
x=5
```

- If no constructor is created, a default constructor initializes instance variables to default values(i.e, zeros).
- A constructor can be parameterized or parameter-less
- Difference between constructor and method in java

Java Constructor	Java Method
Constructor is used to initialize the state of an object.	Method is used to expose behaviour of an object.
Constructor must not have return type.	Method must have return type.
Constructor is invoked implicitly.	Method is invoked explicitly.
The java compiler provides a default constructor if you don't have any constructor.	Method is not provided by compiler in any case.
Constructor name must be same as the class name.	Method name may or may not be same as class name.

Creating an Object:

As mentioned previously, a class provides the blueprints for objects. So basically an object is created from a class. In Java the new keyword is used to create new objects. There are three steps when creating an object from a class:

- Declaration: A variable declaration with a variable name with an object type.
- Instantiation: The 'new' keyword is used to create the object.
- Initialization: The 'new' keyword is followed by a call to a constructor. This call initializes the new object.

Example of creating an object is given below:

```
public class Puppy {
   public Puppy(String name) {
  // This constructor has one parameter, name.
   System.out.println("Passed Name is :"+ name );
   }
   public static void main(String[]args){
      // Following statement would create an object myPuppy
      Puppy myPuppy =new Puppy("tommy");
      }
   }
```

If we compile and run the above program, then it would produce the following result:

```
Passed Name is: tommy
```

Accessing Instance Variables and Methods:

Instance variables and methods are accessed via created objects. To access an instance variable the fully qualified path should be as follows:

```
/* First create an object */
ObjectReference = new Constructor();
/* Now call a variable as follows */
ObjectReference.variableName;
/* Now you can call a class method as follows */
```

```
ObjectReference.MethodName();
```

Example: This example explains how to access instance variables and methods of a class:

```
public class Puppy {
   int puppyAge;
   public Puppy(String name) {
      // This constructor has one parameter, name.
      System.out.println("Passed Name is :" + name);
   }
   public void setAge(int age) {
      puppyAge = age;
   }
   public int getAge() {
      System.out.println("Puppy's age is :" + puppyAge);
      return puppyAge;
   }
   public static void main(String[] args) {
      /* Object creation */
      Puppy myPuppy = newPuppy("tommy");
      /* Call class method to set puppy's age */
      myPuppy.setAge(2);
      /* Call another class method to get puppy's age */
      myPuppy.getAge();
      /* You can access instance variable as follows as well */
      System.out.println("Variable Value :" + myPuppy.puppyAge);
   }
}
```

If we compile and run the above program, then it would produce the following result:

```
PassedName is:tommy
Puppy's age is :2
Variable Value :2
```

- **Method Overloading:**

When a class has two or more methods by same name but different parameters, it is known as method overloading. It is different from overriding. In overriding a method has same method name, type, number of parameters etc. if number of parameters are same, then type of parameters should be different.

Lets consider the example shown before for calculating sum of integer type. If, lets say we want to calculate sum oof double type. Then the concept of Overloading will be introduced to create two or more methods with the same name but different parameters. The below example explains the same:

```
//program for method overloading
class Overloading {
   int a = 12, b = 16;
   void add() {
```

```
        System.out.println("Sum of " + a + " and " + b + "=" + (a + b));
      }
      void add(int x, int y) {
        System.out.println("Sum of " + x + " and " + y + "=" + (x + y));
      }
      double add(double x) {
        return x + a + b;
      }
      public static void main(String args[]) {
        Overloading ol = new Overloading();
        ol.add();
        ol.add(16, 28);
        System.out.println("Sum =" + ol.add(1.5));
      }
    }
```

This would produce the following result:

```
Sum of 12 and 16=28
Sum of 16 and 28=44
Sum =29.5
```

- **Using Command-Line Arguments:**

Sometimes you will want to pass information into a program when you run it. This is accomplished by passing command-line arguments to main( ). A command-line argument is the information that directly follows the program's name on the command line when it is executed. To access the command-line arguments inside a Java program is quite easy. they are stored as strings in the String array passed to main( ).

Example: The following program displays all of the command-line arguments that it is called with:

```
public class CommandLine {
  public static void main(String args[]) {
    for (int i = 0; i < args.length; i++) {
      System.out.println("args[" + i + "]: " + args[i]);
    }
  }
}
```

Try executing this program as shown here:

```
>javac CommandLine.java
>java CommandLine 12 GMRIT 16.12
 args[0]:12
 args[1]:GMRIT
 args[2]: 16.12
```

- **Constructor Overloading**:

In Java, a constructor is just like a method but without return type. It can also be overloaded like Java methods.

Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a

different task. They are differentiated by the compiler by the number of parameters in the list and their types.

Example:

```
class COverloading {
   int a, b;
   COverloading() {
      a = 12;
      b = 16;
   }
   COverloading(int x, int y) {
      a = x;
      b = y;
   }
   COverloading(COverloading obj1, COverloading obj2) {
      a = obj1.a + obj2.a;
      b = obj1.b + obj2.b;
   }
   void display() {
      System.out.println("Sum=" + (a + b));
   }
}
class ConOverloading {
   public static void main(String args[]) {
      COverloading ol = new COverloading();
      System.out.println("Default Constructor Calling....!");
      ol.display();
      COverloading ol1 = new COverloading(6, 8);
      System.out.println("Parameterized Constructor Calling....!");
      ol1.display();
      COverloading ol2 = new COverloading(ol, ol1);
      System.out.println("Copy Constructor Calling....!");
      ol2.display();
   }
}
```

This would produce the following result:

```
Default Constructor Calling....!
Sum=28
Parameterized Constructor Calling....!
Sum=14
Copy Constructor Calling....!
Sum=42
```

- **"this" keyword:**
  - 'this' keyword acts like an object reference to current calling object.
  - It is widely used in the cases where name of a local variable becomes same as that of an instance variable. In that case the local variable hides the instance variable. To invoke the instance variable, 'this' can be used to refer to the calling object

Example:

```
int x ;
Sample(int x)
{
x=x; // local variable x gets initialized to itself
}
  'this' can be used as below
int x ;
Sample(int x)
{
this.x=x; // this.x is the instance variable
}
```

Sample Program for ""this" keyword

```
//program for this keyword
class SampleDemo {
   int x, y;
   SampleDemo(int x, int y) {
      this.x = x;
      this.y = y;
   }
   void display(int z) {
      System.out.println("Add1=" + (x + z));
      System.out.println("Add2=" + (y + z));
   }
}
class MainClass1 {
   public static void main(String args[]) {
      SampleDemo sd = new SampleDemo(12, 16);
      sd.display(28);
   }
}
```

This would produce the following result:

```
Add1=40
Add2=44
```

- **Access Modifiers in java**: There are two types of modifiers in java: access modifiers and non-access modifiers. The access modifiers in java specifies accessibility (scope) of a data member, method, constructor or class. There are 4 types of java access modifiers:
  1. private
  2. default
  3. protected
  4. public

Access specifier/ Availability location	private	default access	protected	public
Same class	√	√	√	√
Subclass in the same package	x	√	√	√
Non-subclass in same package	x	√	√	√
Subclass in other packages	x	x	√	√
Non-subclass in other packages	x	x	x	√

--- a member declared 'private' is available only within the class

--- a member declared with no access specifier (default access) is available within the package only

--- a member declared 'protected' is available within the package and also to subclasses declared outside the package

--- a member declared 'public' is accessible by any code anywhere

- **private access modifier**
  The private access modifier is accessible only within class.
  example of private access modifier
  In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is compile time error.

```
class A{
private int data=40;
private void msg(){System.out.println("Hello java");}
}

public class Simple{
 public static void main(String args[]){
   A obj=new A();
   System.out.println(obj.data);//Compile Time Error
   obj.msg();//Compile Time Error
   }
}
```

  Note: A class cannot be private or protected except nested class.
- **default access modifier**
  If you don't use any modifier, it is treated as default bydefault. The default modifier is accessible only within package.
  Example of default access modifier
  In this example, we have created two packages pack and mypack. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside the package.

```
//save by A.java
package pack;
class A{
 void msg(){System.out.println("Hello");}
```

```
}
//save by B.java
package mypack;
import pack.*;
class B{
  public static void main(String args[]){
   A obj = new A();//Compile Time Error
   obj.msg();//Compile Time Error
  }
}
```

In the above example, the scope of class A and its method msg() is default so it cannot be accessed from outside the package.

- **protected access modifier**

  The protected access modifier is accessible within package and outside the package but through inheritance only. The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

  Example of protected access modifier

  In this example, we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only through inheritance.

```
//save by A.java
package pack;
public class A{
protected void msg(){System.out.println("Hello");}
}
//save by B.java
package mypack;
import pack.*;
class B extends A{
  public static void main(String args[]){
   B obj = new B();
   obj.msg();
  }
}
```

  Output: Hello

- **public access modifier**

  The public access modifier is accessible everywhere. It has the widest scope among all other modifiers.

  Example of public access modifier

```
//save by A.java
package pack;
public class A{
public void msg(){System.out.println("Hello");}
}
//save by B.java
package mypack;
import pack.*;
```

```
class B{
  public static void main(String args[]){
   A obj = new A();
   obj.msg();
  }
}
```

Output: Hello

Understanding all java access modifiers

Let's understand the access modifiers by a simple table.

- **Strings**:

Java string is a sequence of characters. They are objects of type String. Once a String object is created it cannot be changed. Stings are Immutable. To get changeable strings use the class called StringBuffer. String and StringBuffer classes are declared final, so there cannot be subclasses of these classes. The default constructor creates an empty string.

```
String s = new String();
```

Creating Strings:

```
String str = "abc"; is equivalent to:
char data[] = {'a', 'b', 'c'};
String str = new String(data);
```

If data array in the above example is modified after the string object str is created, then str remains unchanged.Construct a string object by passing another string object.

```
String str2 = new String(str);
```

- **String Operations:**
  - The **length()** method returns the length of the string.
    Eg: System.out.println("Hello".length()); // prints 5
  - The **+ operator** is used to concatenate two or more strings.
    Eg: String myname = "Harry"
    String str = "My name is" + myname+ ".";
    For string concatenation the Java compiler converts an operand to a String whenever the other operand of the + is a String object. Characters in a string can be extracted in a number of ways.
  - public char **charAt(int index)**
    Returns the character at the specified index. An index ranges from 0 to length() - 1. The first character of the sequence is at index 0, the next at index 1, and so on, as for array indexing.

```
char ch;
ch = "abc".charAt(1); // ch = "b"
```

- **getChars()** - Copies characters from this string into the destination character array.
  public void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)
  – srcBegin - index of the first character in the string to copy.
  – srcEnd - index after the last character in the string to copy.
  – dst - the destination array.
  – dstBegin - the start offset in the destination array.
- **equals()** - Compares the invoking string to the specified object. The result is true if and only if the argument is not null and is a String object that represents the same sequence of characters as the invoking object.

public boolean equals(Object anObject)

- **equalsIgnoreCase()**- Compares this String to another String, ignoring case considerations. Two strings are considered equal ignoring case if they are of the same length, and corresponding characters in the two strings are equal ignoring case.
  public boolean equalsIgnoreCase(String anotherString)
- **startsWith()** – Tests if this string starts with the specified prefix.
  public boolean startsWith(String prefix)
  Eg: "Figure".startsWith("Fig"); // true
- **endsWith()** - Tests if this string ends with the specified suffix.
  public boolean endsWith(String suffix)
  Eg: "Figure".endsWith("re"); // true
- **startsWith()** -Tests if this string starts with the specified prefix beginning at a specified index.
  public boolean startsWith(String prefix, int toffset)
  prefix - the prefix.
  toffset - where to begin looking in the string.
  "figure".startsWith("gure", 2); // true
- **compareTo()** - Compares two strings lexicographically.
  – The result is a negative integer if this String object lexicographically precedes the argument string.
  – The result is a positive integer if this String object lexicographically follows the argument string.
  – The result is zero if the strings are equal.
  – compareTo returns 0 exactly when the equals(Object) method would return true.
  public int compareTo(String anotherString)
  public int compareToIgnoreCase(String str)
- **indexOf()** – Searches for the first occurrence of a character or substring. Returns -1 if the character does not occur.
  public int **indexOf(char ch)**- Returns the index within this string of the first occurrence of the specified character.
  public int **indexOf(String str)** - Returns the index within this string of the first occurrence of the specified substring.
  Eg: String str = "How was your day today?";
  str.indexof('t');
  str("was");
  public int **indexOf(int ch, int fromIndex)**- Returns the index within this string of the first occurrence of the specified character, starting the search at the specified index.
  public int **indexOf(String str, int fromIndex)** - Returns the index within this string of the first occurrence of the specified substring, starting at the specified index.
  String str = "How was your day today?";
  str.indexof("a", 6);
  str("was", 2);
- **lastIndexOf()** –Searches for the last occurrence of a character or substring. The methods are similar to indexOf().

- **substring()** - Returns a new string that is a substring of this string. The substring begins with the character at the specified index and extends to the end of this string.
  public String substring(int beginIndex)
  Eg: "unhappy".substring(2) returns "happy"
  • public String substring(int beginIndex, int endIndex)
  Eg: "smiles".substring(1, 5) returns "mile"
- **concat()** - Concatenates the specified string to the end of this string. If the length of the argument string is 0, then this String object is returned. Otherwise, a new String object is created, containing the invoking string with the contents of the str appended to it.
  public String concat(String str)
  "to".concat("get").concat("her") returns "together"
- **replace()**- Returns a new string resulting from replacing all occurrences of oldChar in this string with newChar.
  public String replace(char oldChar, char newChar)
  "mesquite in your cellar".replace('e', 'o') returns "mosquito in your collar"
- **trim()** - Returns a copy of the string, with leading and trailing whitespace omitted.
  public String trim()
  String s = "Hi Mom! ".trim();
  S = "HiMom!"
- **valueOf()** – Returns the string representation of the char array argument.
  public static String valueOf(char[] data)
  The contents of the character array are copied; subsequent modification of the character array does not affect the newly created string.
  Other forms are:
  public static String valueOf(char c)
  public static String valueOf(boolean b)
  public static String valueOf(int i)
  public static String valueOf(long l)
  public static String valueOf(float f)
  public static String valueOf(double d)
- **toLowerCase():** Converts all of the characters in a String to lower case.
- **toUpperCase():** Converts all of the characters in this String to upper case.
  public String toLowerCase()
  public String toUpperCase()
  Eg: "HELLO THERE".toLowerCase();
  "hello there".toUpperCase();

➢ **StringBuffer :**

A StringBuffer is like a String, but can be modified. The length and content of the StringBuffer sequence can be changed through certain method calls.

• StringBuffer defines three constructors:
– StringBuffer()
– StringBuffer(int size)
– StringBuffer(String str)

➢ **StringBuffer operations:**

• The principal operations on a StringBuffer are the append and insert methods, which are overloaded so as to accept data of any type.

Here are few append methods:

StringBuffer **append(String str)**

StringBuffer **append(int num)**

• The append method always adds these characters at the end of the buffer.

• The insert method adds the characters at a specified point.

Here are few insert methods:

StringBuffer **insert(int index, String str)**

StringBuffer **insert(int index, char ch)**

Index specifies at which point the string will be inserted into the invoking StringBuffer object.

- **delete()** - Removes the characters in a substring of this StringBuffer. The substring begins at the specified start and extends to the character at index end - 1 or to the end of the StringBuffer if no such character exists. If start is equal to end, no changes are made.

  public StringBuffer **delete(int start, int end)**
- **replace()** - Replaces the characters in a substring of this StringBuffer with characters in the specified String.

  public StringBuffer **replace(int start, int end, String str)**
- **substring()** - Returns a new String that contains a subsequence of characters currently contained in this StringBuffer. The substring begins at the specified index and extends to the end of the StringBuffer.

  public String **substring(int start)**
- **reverse()** - The character sequence contained in this string buffer is replaced by the reverse of the sequence.

  public String **reverse()**
- **length()** - Returns the length of this string buffer.

  public int **length()**
- **capacity()** - Returns the current capacity of the String buffer. The capacity is the amount of storage available for newly inserted characters.

  public int **capacity()**
- **charAt()** - The specified character of the sequence currently represented by the string buffer, as indicated by the index argument, is returned.

  public char **charAt(int index)**
- **getChars()** - Characters are copied from this string buffer into the destination character array dst. The first character to be copied is at index srcBegin; the last character to be copied is at index srcEnd-1.

  public void **getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)**
- **setLength()** - Sets the length of the StringBuffer.

  public void **setLength(int newLength)**

  Examples:

  StringBuffer sb = new StringBuffer("Hello");

  sb.length(); // 5

  sb.capacity(); // 21 (16 characters room is added if no size is specified)

  sb.charAt(1); // e

```
sb.setCharAt(1,'i'); // Hillo
sb.setLength(2); // Hi
sb.append("l").append("l"); // Hill
sb.insert(0, "Big "); // BigHill
 sb.replace(3, 11, " "); // Big
sb.reverse(); // giB
```

- **Garbage collection:**

In java, garbage means unreferenced objects. Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects. To do so, we were using free() function in C language and delete() in C++. But, in java it is performed automatically. So, java provides better memory management.

Advantage of Garbage Collection

- It makes java memory efficient because garbage collector removes the unreferenced objects from heap memory.
- It is automatically done by the garbage collector(a part of JVM) so we don't need to make extra efforts.

How can an object be unreferenced?

There are many ways:

- By nulling the reference
- By assigning a reference to another
- By annonymous object etc.

- **By nulling a reference:**

```
Employee e=new Employee();
e=null;
```

- **By assigning a reference to another:**

```
Employee e1=new Employee();
Employee e2=new Employee();
e1=e2;//now the first object referred by e1 is available for garbage collection
```

- **By annonymous object:**

```
new Employee();
```

- **finalize() method:** The finalize() method is invoked each time before the object is garbage collected. This method can be used to perform cleanup processing. This method is defined in Object class as:

```
protected void finalize(){}
```

Note: The Garbage collector of JVM collects only those objects that are created by new keyword. So if you have created any object without new, you can use finalize method to perform cleanup processing (destroying remaining objects).

- **gc() method:** The gc() method is used to invoke the garbage collector to perform cleanup processing. The gc() is found in System and Runtime classes.

```
public static void gc(){}
```

Note: Garbage collection is performed by a daemon thread called Garbage Collector(GC). This thread calls the finalize() method before object is garbage collected.

Simple Example of garbage collection in java

```
public class TestGarbage1{
 public void finalize(){System.out.println("object is garbage collected");}
 public static void main(String args[]){
  TestGarbage1 s1=new TestGarbage1();
  TestGarbage1 s2=new TestGarbage1();
  s1=null;
  s2=null;
  System.gc();
 }
}
```

Test it Now

object is garbage collected
object is garbage collected

UNIT I - Self Study Course:

Features of object-oriented programming–Java History–Computer Programming Hierarchy–Role of Java Programmer in Industry